

Analysis: Broken Clock

Test Set 1

In Test Set 1, our only unknown is which hand is which. Since there are only $3!$ possibilities for the assignment of angles in the input to a specific hand, we can just try them all. After we have angles assigned to specific hands, we can just read the time. The easiest way is to just check the hours hand, as the statement tells us it moves one tick per nanosecond, so the current number of nanoseconds after midnight is equal to the angle in ticks of the hours hand. Notice that we still need to check that the time in the other two hands is consistent with this! We can do this by reading the number of minutes and seconds from their hands using the definitions in the statements, and see if it is consistent with the current number of minutes and seconds as computed from the nanoseconds we got from the hours hand. Alternatively, we can do the reverse process of getting from current time to how the clock should look. We do that in more detail below, as it is needed to solve the other test sets.

The solution to Test Sets 2 and 3 is based on first solving the inverse problem: given a time of the day, find a canonical set of 3 angles that would produce it. Luckily, the way to solve it is explained on the statement: we know the speeds of each hand so we can translate a time of the day into a number of ticks since midnight, and then take it modulo the number of ticks in a full circle ($12 \times 10^{10} \times 360$). Once we get three angles, we need to consider the potential rotations. We can say that canonical sets of angles always have at least one 0, but there can be up to 3 ways to do that (choosing each hand to be the 0 one): simply choosing the [lexicographically least](#) of them (after sorting them) suffices.

Test Set 2

In Test Set 2, there are only $12 \times 60 \times 60$ possible times of the day to consider. We can simply build the canonical representation a for each time t and store it in a reverse dictionary from representations to times (which one we save if there are multiples does not matter) as a first step. Then, for each input, we get its canonical representation and find a corresponding time in the dictionary to output.

Test Set 3

In Test Set 3, the number of different times of the day is too large to check them all individually. A possible approach is to restrict the number of times of the day that can correspond to a specific input set of angles a and then check each of those to see if anyone fully matches by using the reverse conversion.

We can try every possible assignment of each angle to a specific hand. There are only $3!$ possibilities for that. Let us say that we now have an angle of a_h for the hours hand, a_m of the minutes hand and a_s for the seconds hand.

Let us write the real time as h full hours plus n nanoseconds, that is, $h \cdot 3600 \cdot 10^9 + n$ nanoseconds since midnight, with h between 0 and 11. The angle (in ticks) of the minute hand is $12 \cdot n$ and the angle of the hour hand is $h \cdot 3600 \cdot 10^9 + n$. The difference between those two numbers $h \cdot 3600 \cdot 10^9 - 11 \cdot n$ needs to be equal to the difference between the angles we read from the input $a_h - a_m$. Note that this works because the difference between angles is invariant through rotations. If we try every possible value for h between 0 and 11, every variable except n

in the last equality has an actual value, and we can solve for n . If the value for n happens to be an integer, it gives us a real time candidate.

Finally, for each of the up to $3! \cdot 12$ real time candidates, we do the reverse check as we did in the previous solution.

Analysis: Digit Blocks

Let $D_{i,j}$ be the digit on the block that has exactly j blocks under it in the i -th tower. The number written on the i -th tower is then

$$\sum_{j=0}^{B-1} 10^j \cdot D_{i,j}$$

and the total score is

$$\sum_{i=1}^N \sum_{j=0}^{B-1} 10^j \cdot D_{i,j}.$$

Notice that each term in the last sum is independent from others, that is, each position in a tower has a particular value that gets multiplied by the digits, and exchanging blocks at the same heights that go on different towers does not change the final outcome. Our job to maximize the score is to get the largest digits paired up with the largest values. That means, putting large digits near the top of the towers. Moreover, the value of a position is more than 9 times the value of all the positions below it combined. Therefore, delaying the use of a position until we can get the maximum possible digit can be worth sacrificing the ability to decide on the order of all positions below it for up to 9 towers, and possibly a lot more (as even a random order gets some value).

All the approaches we present are based on trying to reserve high-valued positions for high-valued digits, sacrificing other positions.

Greedy strategies

The simplest greedy strategy is: try to place 9s in the top position of every tower, and sacrifice everything else. That means, when a block comes, if it is a 9, place it in the highest non-finished tower we have. If it is not, place it in the highest tower we have that has fewer than $B - 1$ blocks on it. This reserves the top spots for 9s and accelerates the availability of newer top spots by trying to build towers up as soon as possible.

There are other similar strategies, like doing the same but trying to place 9s in the top two spots of every tower, possibly pivoting to just the top spot when we are running out of time. Similarly, we can pivot to accepting lower digits for second-from-the-top spots or for top spots when there are few blocks left and the likelihood of enough 9s coming becomes low.

These simple strategies oscillate between getting a score of 87% and 91% of S , which means many of them are enough to pass Test Set 1, but none seems to be close to pass Test Set 2. More sophisticated heuristics can pass Test Set 2, but they need to consider at least the top 2 positions and also deal with tactically placing not just 9s but also 8s. These type of solution need careful tweaking and also comes with some uncertainty that it will eventually work out. The solutions in the next section address both those issues, ensuring the points and quite possibly saving us time.

[Dynamic programming](#) strategies

An alternative way is to try to maximize the expected score. This does not exactly maximize the probability of getting above a specific threshold (being more aggressive or conservative near the end depending on how close you actually are to the threshold might be better), but it's really close and much simpler.

The optimal such solution (the one that gets S as its expected score) is too slow, but it is worthwhile to consider it as a starting point. By [linearity of expectation](#), when considering where to put a block, only the heights of all current towers matters, but not the actual values that were put there (since that is the score we will get regardless of all future decisions). We can therefore consider the multiset of tower heights the status, and optimize a function $f(\text{status}, \text{next_digit})$ that checks every possible placement for each possible next digit. The problem with this idea is, of course, the status space is too big for the time limit. This is what we did to calculate S , but we ran it for a lot longer than the time limit.

As we mentioned before, we can improve on the time limit by not caring too much about what happens with the low-value positions. That is, consider only a subset of the statuses. One example is this: for each digit we consider only the options of the first greedy strategy. We either put it in the top-most position of some tower, or we put it in the highest tower that has fewer than $B - 1$ blocks on it. This severely reduces the number of possible statuses, as there is only one tower that can have a height other than 0, $B - 1$ or B . Instead of making the decision greedily, we leave the decision of which option to choose to the dynamic programming part. This solution is fast enough and gets about 96% of S as its score. Not enough for Test Set 2, but getting closer. Doing the same but leaving the top 2 positions up for optimization, on the other hand, gets over 99.5% of S and passes Test Set 2.

Since the top position represents close to 90% of the value, it makes sense that optimizing it gets 90% of the score. Similarly, the top two positions represent close to 99% of the value. More formally, consider two modified problems in which the top position or top two positions retain their value, but all other positions are valued as 0. Our proposed solutions get the maximum expected scores S_1 and S_2 for those modified problems, and we know that $S_1 > 0.9 \cdot S$ and $S_2 > 0.99 \cdot S$. Therefore, our solutions' expected score is guaranteed to be above each respective threshold, and the extra score we get from the lower positions gives us an additional gap to make the probability of success extremely high: our estimates are that each solution has less than 10^{-10} probability of failing the respective test set.

Analysis: Subtransmutation

To solve this problem, we can first tackle a simplified version: given a unit of metal x , can it be used to produce all the metals required by the input?

To solve this simplified problem, we can use a greedy strategy. Keep track of the multiset H of units of metal we have (starting with $\{x\}$) and the multiset D of units of metal we owe (starting with the ones given as input). Iterate the following steps:

1. Remove the intersection $H \cap D$ from both H and D .
2. If D is empty, the answer is yes.
3. If H is empty and D is not, the answer is no.
4. Take all c units of the metal i with the largest number still in H . Remove all c units of i from H and insert c units of $i - \mathbf{A}$ and c units of $i - \mathbf{B}$ into H (if either new metal number is invalid, skip those).

This procedure works because those units that we greedily transform are not something that we owe, so we have nothing better to do with them (it may be futile to convert them, but it does not hurt). And since we have to produce the units that we owe, we might as well just pay them back as soon as possible, as anything we could do with the current unit we could also do with a future unit that we would use to pay the debt instead.

Test Set 1

In Test Set 1, notice that if it is possible to find an answer m for an input where $\mathbf{N}$ and all $\mathbf{U}_i$ have maximum value, that procedure also produces enough units for any other possible input. If we implement the greedy strategy above and try it for this particular input on increasingly large values for m , we can quickly arrive at the realization that $m = 29$ solves it. This means that all inputs can be generated with a metal with number no greater than 29. So, using the same procedure of checking increasingly large values of m for the given input is a valid solution for the full Test Set.

It is also possible to notice and/or prove theoretically that there are no impossible cases and that the answers are small. A quick argument is that numbers grow exponentially: from a single unit of number m we can create 2 units of number $m - 2$ (with leftovers). This means we can create 2^i units of element $m - 2i$ from a single unit of element m . We can distribute these among the elements $m - 2i - \mathbf{N} + 1$ through $m - 2i$ (with a lot of leftovers), getting $2^i / \mathbf{N}$ units of each of $\mathbf{N}$ consecutive elements. These can be further moved to elements with lower numbers if needed. This shows that an m such that $2^{m-2 \times 20} / 20 > 20$ ought to be enough, and $m = 49$ fulfills such a requirement.

Test Set 2

As the samples show, impossible cases can happen in Test Set 2. This means running our Test Set 1 solution as is will result in the program not finishing and getting a Time Limit Exceeded error. A simple solution, however, is to notice that there is also a somewhat small maximum answer for all possible cases, and then simply stop once we are past it and return impossible.

Knowing what that maximum answer is, and proving it, is harder as we cannot just rely on experiments. We have to lean more on the theory of the problem.

We can first notice that starting from metal m , every metal we can possibly produce has the same remainder as m modulo the [greatest common divisor](#) between $\mathbf{A}$ and $\mathbf{B}$ (let us call that g). This is an application of [Diophantine equations](#), but while knowing that specific theory could help tackle the problem, it is not absolutely necessary. Now, consider the remainder modulo g of every i such that $\mathbf{U}_i > 0$. If they are all the same value k , then m modulo g needs to be k as well. If they are not all the same, then the case is impossible.

At this point, we can express all metal numbers that matter as $g \times x + k$ for some x . To show that all of remaining cases are possible, we can generalize the proof we did for Test Set 1 and use [Bézout's identity](#) to build them, or we can simply iterate all relatively prime values for $\mathbf{A}/g$ and $\mathbf{B}/g$ and solve the largest case experimentally. If we do that, we can arrive at the conclusion that the maximum possible answer for a possible Test Set 2 case is 402. Therefore, we can do the same solution as in Test Set 1, exploring values for m up to 402, and returning impossible if we did not find an answer.